

SCHOOL OF COMPUTER SCIENCES  
UNIVERSITI SAINS MALAYSIA

CPT212 : Design & Analysis Algorithm  
Semester II, Academic Session: 2025/2026

Assignment 1 : Report  
Lecturer : Assoc Prof. Dr. Tan Tien Ping

| Student Name                          | Matric No |
|---------------------------------------|-----------|
| Haris Zakuwan Bin Sa-udi              | 23302870  |
| Danish Shazwan Bin Muhammad Khusyairi | 22306866  |

Date of submission:  
3rd May 2026

Github Repository : <https://github.com/Skywalkz1/CPT212-Assignment1.git>

## TABLE OF CONTENTS

|                                                     |   |
|-----------------------------------------------------|---|
| TABLE OF CONTENTS.....                              | 1 |
| 1. Introduction.....                                | 2 |
| 2. Objective.....                                   | 2 |
| 2.1 Primary Objectives.....                         | 2 |
| 2.2 Specific Operational Objectives.....            | 2 |
| 3. Theoretical Complexity Analysis.....             | 2 |
| 3.1 Simple (Standard) Multiplication Algorithm..... | 2 |
| 3.2 Karatsuba Algorithm.....                        | 3 |
| 3.3 Summary.....                                    | 3 |
| 4. Experimental Setup & Methodology.....            | 3 |
| 4.1 Data Structure and Large Number Handling.....   | 4 |
| 4.2 Dataset Generation.....                         | 4 |
| 5. Experimental Results.....                        | 4 |
| 6.0 Graphical Representation.....                   | 5 |
| 6.1 Performance Comparison.....                     | 6 |
| 7. Conclusion.....                                  | 7 |
| 8. Output Sample.....                               | 8 |
| 9.0 Appendix.....                                   | 9 |

## 1. Introduction

This report presents a comparative analysis of the Simple Multiplication and Karatsuba algorithms implemented in Java. To fulfill the primary objectives of the assignment, both methods were engineered to process massive integers up to 10,000 digits. By tracking primitive operations, this study contrasts experimental execution costs against theoretical Big-O time complexities, ultimately demonstrating Karatsuba's superior asymptotic efficiency at scale.

## 2. Objective

### 2.1 Primary Objectives

The foundational goals of this assignment, as outlined by the course specification, are:

- To manipulate data structures or algorithms in problem-solving and programming.
- To perform a theoretical and empirical complexity analysis of algorithms.

### 2.2 Specific Operational Objectives

To successfully fulfill the primary goals, this project aims to achieve the following specific outcomes:

- **Algorithmic Implementation:** To implement both the standard Simple Multiplication algorithm and the Karatsuba algorithm in Java, ensuring the data structures chosen can accurately process and compute extremely large integers up to  $n = 10000$  digits.
- **Performance Instrumentation:** To instrument both Java implementations with internal counters designed to accurately track and record the number of primitive operations executed during computation.
- **Empirical Experimentation:** To conduct a scalable experiment by randomly generating pairs of  $n$ -digit numbers, recording the operation counts, and plotting a graph to visualize the relationship between input size ( $n$ ) and computational cost.
- **Comparative Analysis:** To mathematically derive the theoretical Big-O time complexity for both algorithms and critically compare these theoretical bounds against the plotted experimental results, culminating in a discussion on algorithmic efficiency at scale.

## 3. Theoretical Complexity Analysis

To accurately evaluate the performance of both algorithms, a theoretical baseline must first be established.

### 3.1 Simple (Standard) Multiplication Algorithm

The standard multiplication algorithm operates on a fundamental "digit-by-digit" basis. When multiplying two numbers, each consisting of  $n$  digits, the algorithm employs a nested

iteration structure. The outer loop iterates through the  $n$  digits of the multiplier, and for each of these digits, an inner loop iterates through the  $n$  digits of the multiplicand. Because a constant number of primitive operations occurs within this inner loop, the total number of operations scales proportionally to  $n \times n$ . Consequently, the theoretical time complexity is strictly quadratic, expressed as  $O(n^2)$ .

### 3.2 Karatsuba Algorithm

Unlike the simple method, the Karatsuba algorithm utilizes a recursive "divide-and-conquer" approach. Instead of calculating every cross-product, it splits the  $n$ -digit operands into two halves (of size  $n/2$ ) and uses algebraic substitution to compute the result using only three recursive multiplications instead of the traditional four.

The time complexity can be derived using the Master Theorem for divide-and-conquer recurrences:

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

- $a = 3$  : The number of recursive sub-problems ( Calculating  $z_0, z_1, z_2$  )
- $b = 2$  : The size of each sub-problem relative to the original ( $n/2$ ).
- $O(n)$ : The linear time taken to perform the additions, subtractions, and positional shifts outside the recursive calls.

Because  $\log_2(3) \approx 1.585$  , the recurrence is leaf-dominated. Therefore, the theoretical time complexity is bounded at  $O(n^{\log_2 3})$ , which is  $O(n^{1.585})$

### 3.3 Summary

Theoretically, while Simple Multiplication  $O(n^2)$  may perform adequately for very small inputs due to low systemic overhead , the Karatsuba algorithm  $O(n^{1.585})$  is superior. As  $n$  approaches large scales, Karatsuba will require drastically fewer primitive operations.

## **4. Experimental Setup & Methodology**

Both algorithms were implemented using the Java programming language with standard Java SE libraries. To handle very large integers (up to 10,000 digits), the BigInteger class was utilized, particularly for the Karatsuba implementation to ensure numerical accuracy without overflow.

To measure performance, internal counters were embedded within each algorithm to track primitive operations such as assignments, arithmetic computations, and object creations.

These counters provide a consistent metric for evaluating computational cost beyond just execution time.

For testing, pairs of random base-10 integers were generated for varying input sizes ranging from  $n = 10$  up to  $n = 10,000$  digits. Each pair was processed by both algorithms, and the corresponding operation counts were recorded. The collected data was then visualized using line graphs to illustrate how the number of operations scales with input size.

#### 4.1 Data Structure and Large Number Handling

Standard Java primitive data types, such as `int` and `long`, suffer from buffer overflow when processing integers larger than 18 digits. To satisfy the requirement of handling massive inputs up to  $n=10000$ , both algorithms were engineered utilizing the `java.math.BigInteger` class.

- For the **Simple Multiplication** algorithm, the `BigInteger` inputs were converted into arrays of integers (`int[]`). This allowed the algorithm to isolate single digits and accurately compute specific partial products and carriers at every step, directly fulfilling the algorithmic specification
- For the **Karatsuba algorithm**, `BigInteger`'s native `.divide()` and `.remainder()` methods were utilized to safely split the massive  $n$ -digit numbers into halves without data loss or overflow.

#### 4.2 Dataset Generation

To conduct a fair and scalable experiment, a custom dataset was procedurally generated. A helper function was created utilizing Java's `java.util.Random` class to construct random  $n$ -digit numbers. To ensure the experiment is strictly reproducible and scientifically valid, a fixed procedural seed (`Random(42)`) was implemented. The dataset included input sizes ranging from small numbers ( $n = 10, 50, 100$ ) to identify the algorithmic crossover point, up to massive scales ( $n = 1000, 5000, 10000$ ) to observe asymptotic growth.

### 5. Experimental Results

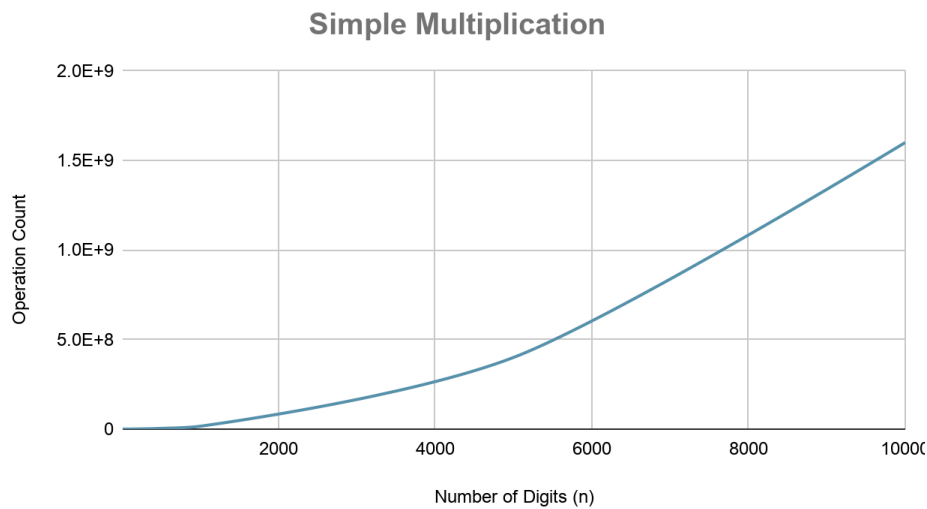
| Number of Digits (n) | Simple Multiplication | Karatsuba Multiplication |
|----------------------|-----------------------|--------------------------|
| 10                   | 1886                  | 4360                     |
| 50                   | 41350                 | 53481                    |
| 100                  | 1622666               | 163525                   |
| 500                  | 4013318               | 2206468                  |

|       |            |           |
|-------|------------|-----------|
| 1000  | 16026604   | 6636053   |
| 5000  | 400132972  | 83299857  |
| 10000 | 1600265998 | 250141614 |

*Table 1: Operation Counts for each method compared with Number of Digits (n)*

The experimental results obtained from running both algorithms are summarized in Table 1 and illustrated in the corresponding graphs. The data clearly shows that as the number of digits increases, the number of operations required by both algorithms also increases, but at significantly different rates. The Simple Multiplication algorithm exhibits a very steep growth pattern, while the Karatsuba algorithm increases at a comparatively slower pace.

## 6.0 Graphical Representation



*Figure 1: Simple Multiplication Line Chart (Operation Count against Number of Digits , n)*

The Simple Multiplication algorithm demonstrates a rapid quadratic growth pattern. As input size increases, the number of operations grows dramatically, reaching over 1.6 billion operations at  $n = 10,000$ . This aligns with its theoretical time complexity of  $O(n^2)$ , where performance degrades quickly as input size scales.

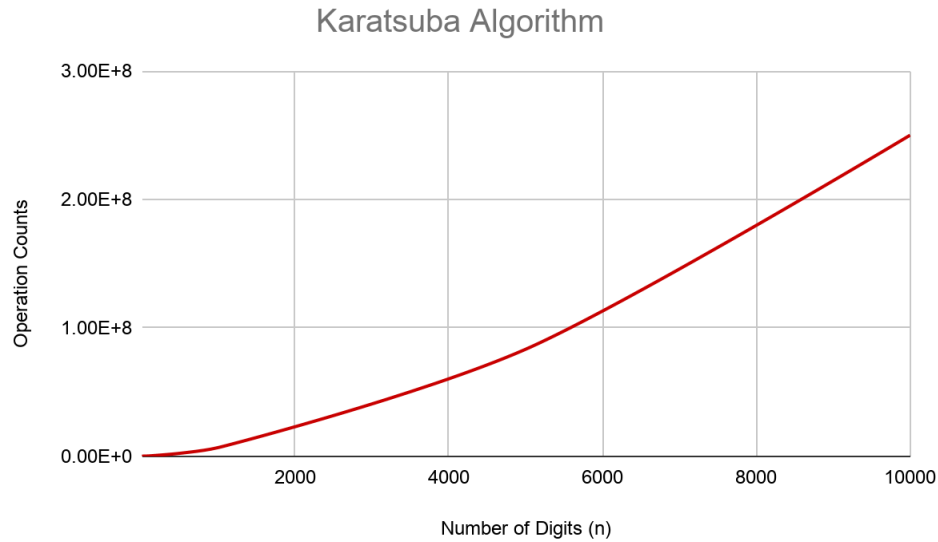


Figure 2: Karatsuba Algorithm Line Chart (Operation Count against Number of Digits,  $n$ )

In contrast, the Karatsuba algorithm shows a much slower increase in operation count. Although it still grows with input size, the curve is noticeably less steep. This behavior is consistent with its sub-quadratic complexity of approximately  $O(n^{1.585})$ , making it significantly more efficient for large inputs.

### 6.1 Performance Comparison

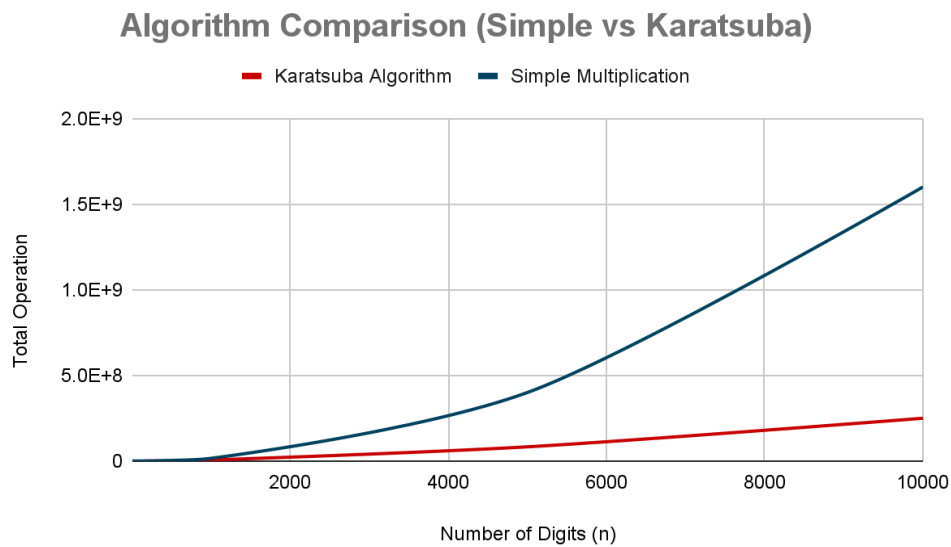


Figure 3: Comparison Line Chart (Operation Count against Number of Digits,  $n$ )

While both algorithms correctly compute the multiplication results, their efficiency differs greatly. For smaller input sizes, the difference in performance is less pronounced, and in some cases, the Simple Multiplication algorithm may even appear competitive due to lower overhead.

However, as the input size increases, the advantage of Karatsuba becomes evident. At  $n = 10,000$  digits, the Simple Multiplication algorithm requires several times more operations than Karatsuba. This highlights the scalability issue of the standard approach and the effectiveness of divide-and-conquer optimization.

## 7. Conclusion

This study successfully implemented and analyzed two different multiplication algorithms: Simple Multiplication and Karatsuba. Through both theoretical derivation and experimental evaluation, it is evident that the two approaches exhibit fundamentally different performance characteristics.

The Simple Multiplication algorithm follows a quadratic time complexity  $O(n^2)$ , which leads to rapid growth in computational cost as input size increases. On the other hand, the Karatsuba algorithm achieves a more efficient time complexity of approximately  $O(n^{1.585})$  by reducing the number of required multiplications through a divide-and-conquer strategy.

The experimental results closely match the theoretical predictions, confirming that Karatsuba is significantly more efficient for large-scale integer multiplication. Therefore, for applications involving very large numbers, the Karatsuba algorithm is a more suitable and scalable choice compared to the standard method.

## 8. Output Sample

```

=====
DEMO: 52301 x 380    (from PDF)
=====
partial products for (52301 x 0): 000000
carriers for        (52301 x 0): 000000
partial products for (52301 x 8): 418408
carriers for        (52301 x 8): 041200
partial products for (52301 x 3): 156903
carriers for        (52301 x 3): 010000
Result   : 19874380
Expected : 19874380
Correct  : true
Total operations : 341

=====
DEMO: 1234 x 5678
=====
partial products for (1234 x 8): 09872
carriers for        (1234 x 8): 00123
partial products for (1234 x 7): 08638
carriers for        (1234 x 7): 00122
partial products for (1234 x 6): 07404
carriers for        (1234 x 6): 00122
partial products for (1234 x 5): 06170
carriers for        (1234 x 5): 00112
Result   : 7006652
Expected : 7006652
Correct  : true
Total operations : 374

=====
EMPIRICAL EXPERIMENT - Operation Counts for Random n-Digit Inputs
=====
n, opCount
10, 1886
50, 41350
100, 162666
500, 4013318
1000, 16026604
5000, 400132972
10000, 1600265998

Theoretical time complexity: O(n^2)
Justification: two nested loops, each running n iterations,

```

*Image 1 : Output Example of Part 1 ( Simple Multiplication )*

```

=====
Karatsuba Correctness Verification
=====
1234 x 5678   karatsuba=7006652   expected=7006652   OK=true   ops=993
102 x 313     karatsuba=31926     expected=31926     OK=true   ops=312
1345 x 63456  karatsuba=85348320  expected=85348320  OK=true   ops=1211
52301 x 380   karatsuba=19874380  expected=19874380  OK=true   ops=993
356281 x 997389 karatsuba=355350750309 expected=355350750309 OK=true   ops=2023
=====
Part 2: Empirical Comparison
=====
n, simpleOps, karatsubaOps
10, 1886, 4360
50, 41350, 53481
100, 162666, 163525
500, 4013318, 2206468
1000, 16026604, 6636053
5000, 400132972, 83299857
10000, 1600265998, 250141614

=== Complexity Summary ===
Simple Multiplication : O(n^2)
Karatsuba Algorithm   : O(n*log2(3)) ~ O(n^1.585)

Empirical observation:
For small n, Karatsuba has more overhead than Simple.
Beyond ~n=100 digits, Karatsuba's op count grows noticeably
slower, confirming its theoretical superiority at large n.

[Done] exited with code=0 in 201.421 seconds

```

*Image II : Output Example of Part 2 ( Karatsuba Multiplication Algorithm )*

## 9.0 Appendix

Dataset ( Simple vs Karatsuba )

[https://docs.google.com/spreadsheets/d/1BUfHZV\\_Fu1pFPVHQ8EntEtAXedRY4ALjTxSOBXvVfBo/edit?usp=sharing](https://docs.google.com/spreadsheets/d/1BUfHZV_Fu1pFPVHQ8EntEtAXedRY4ALjTxSOBXvVfBo/edit?usp=sharing)

Github Repository ( Code )

<https://github.com/Skywalkz1/CPT212-Assignment1.git>